

Analysis: Results 2026

[Code ▼](#)

- Hands-On Workshop: Navigating the Big Data Analytics Lifecycle with R
 - Duration: ~2 hours | Level: Intermediate | Language: R (tidyverse / tidymodels)
 - Welcome!
-  Concept explanation (theory + business relevance)
 - Phase 1: Data Preparation & Hygiene (25 min)
 -  Concept
 -  Worked Example: Handle Missing Values & Filter Invalid Rows
 - Phase 2: Model Planning & Exploratory Data Analysis (20 min)
 -  Concept
 -  Worked Example: Tenure Distribution by Churn
 - Phase 3: Model Building & Validation (25 min)
 -  Concept
 -  Worked Example: Compare Confusion Matrices
 - Phase 4: Communicating Results & Findings (20 min)
 -  Concept
 -  Worked Example: Executive Summary
 - Phase 5: Operationalizing Analytics Models (15 min)
 -  Concept
 -  Packaging as an API with `plumber`
 -  A Note on Monitoring & Concept Drift
 -  Workshop Wrap-Up

Hands-On Workshop: Navigating the Big Data Analytics Lifecycle with R

Duration: ~2 hours | Level: Intermediate | Language: R (tidyverse / tidymodels)

Welcome!

In this workshop we will walk through the Big Data Analytics Lifecycle end-to-end using a realistic business dataset: the Telco Customer Churn dataset. You will move through five phases that mirror how data science projects are run in industry:

Phase Focus Time
1 Data Preparation & Hygiene 25 min
2 Model Planning & EDA 20 min
3 Model Building & Validation 25 min
4 Communicating Results & Findings 20 min
5 Operationalizing Analytics Models 15 min

Each phase includes:

Concept explanation (theory + business relevance)

 Worked example (fully runnable code — just execute each cell in order) Prerequisites

Basic familiarity with R and the `%>%` / `|>` pipe Basic understanding of classification models (logistic regression, decision trees) No prior tidymodels experience required — we'll explain as we go Dataset

We'll use the IBM Telco Customer Churn dataset, loaded directly from GitHub:

<https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv> (<https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv>)

- It contains ~7,043 customer records with demographic, account, and service-usage attributes, and a Churn label indicating whether the customer left the company.

Setup: Install & Load Packages

Run the cell below once to install any missing packages (skip if already installed in your environment — e.g. most JupyterHub / RStudio Cloud R kernels used in class already have these).

Hide

```
install.packages(c(
  "tidyverse",    # data wrangling & visualization (dplyr, ggplot2, readr, etc.)
  "tidymodels",  # unified modeling framework (recipes, parsnip, rsample, yardstick)
  "corrr",        # tidy correlation matrices + network plots
  "pROC",         # ROC curve / AUC utilities
  #"vip",         # variable importance plots
  "ranger",       # fast random forest engine used by tidymodels
  "plumber"       # turns R scripts into REST APIs (used in Phase 5)
))
```

```
trying URL 'http://rspm/default/__linux__/noble/latest/src/contrib/tidyverse_2.0.0.ta
r.gz'
trying URL 'http://rspm/default/__linux__/noble/latest/src/contrib/tidymodels_1.5.0.t
ar.gz'
trying URL 'http://rspm/default/__linux__/noble/latest/src/contrib/corrr_0.4.5.tar.g
z'
trying URL 'http://rspm/default/__linux__/noble/latest/src/contrib/pROC_1.19.0.1.tar.
gz'
trying URL 'http://rspm/default/__linux__/noble/latest/src/contrib/ranger_0.18.0.tar.
gz'
trying URL 'http://rspm/default/__linux__/noble/latest/src/contrib/plumber_1.3.3.tar.
gz'
```

The downloaded source packages are in
'/tmp/RtmpbsKTii/downloaded_packages'

Hide

```
# ---- Load libraries ----
suppressPackageStartupMessages({
  library(tidyverse)    # dplyr, ggplot2, readr, purrr, tibble ...
  library(tidymodels)   # recipes, parsnip, rsample, workflows, yardstick
  library(corr)         # tidy correlations
  library(pROC)         # ROC / AUC
  # library(vip)        # variable importance plots
  library(ranger)       # random forest engine
})

# Reproducibility
set.seed(2026)

cat("Environment ready. R version:", R.version.string, "\n")
```

```
Environment ready. R version: R version 4.6.1 (2026-06-24)
```

Phase 1: Data Preparation & Hygiene (25 min)

Concept

The Big Data Analytics Lifecycle always begins with Data Preparation — often called ETL (Extract, Transform, Load) or “data hygiene.” Industry surveys consistently show data scientists spend 60–80% of project time here. Why does it matter so much?

Garbage in, garbage out: No model, however sophisticated, can compensate for dirty data. Type errors compound: A column read as text instead of numeric silently breaks every downstream calculation. Missing values bias results: Naively dropping or filling missing data can introduce bias that misleads business decisions (e.g., systematically under-representing a customer segment). Business relevance: For a telecom company, an unclean Churn dataset could lead to a retention model that misidentifies at-risk customers — a direct hit to revenue. Learning goals: By the end of this phase you will be able to:

Load raw data and immediately profile it for quality issues. Diagnose and fix a string-formatted numeric column. Impute missing values responsibly (median imputation) and justify the choice. Encode a categorical target variable for modeling.

```
# ---- Load the raw dataset ----
#data_url <- "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"

library(tidyverse)
churn_raw <- read_csv("Telco-Customer-Churn.csv", show_col_types = FALSE)

glimpse(churn_raw)
```

```

Rows: 7,043
Columns: 21
$ customerID      <chr> "7590-VHVEG", "5575-GNVDE", "3668-QPYBK", "7795-CFOCW", "923
7-H...
$ gender          <chr> "Female", "Male", "Male", "Male", "Female", "Female", "Mal
e", "...
$ SeniorCitizen   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, ...
$ Partner         <chr> "Yes", "No", "No", "No", "No", "No", "No", "No", "No", "Yes", "N
o", "...
$ Dependents      <chr> "No", "No", "No", "No", "No", "No", "Yes", "No", "No", "Ye
s", "...
$ tenure          <dbl> 1, 34, 2, 45, 2, 8, 22, 10, 28, 62, 13, 16, 58, 49, 25, 69,
52,...
$ PhoneService    <chr> "No", "Yes", "Yes", "No", "Yes", "Yes", "Yes", "No", "Yes",
"Ye...
$ MultipleLines   <chr> "No phone service", "No", "No", "No phone service", "No", "Y
es"...
$ InternetService <chr> "DSL", "DSL", "DSL", "DSL", "Fiber optic", "Fiber optic", "F
ibe...
$ OnlineSecurity  <chr> "No", "Yes", "Yes", "Yes", "No", "No", "No", "Yes", "No", "Y
es"...
$ OnlineBackup    <chr> "Yes", "No", "Yes", "No", "No", "No", "Yes", "No", "No", "Ye
s",...
$ DeviceProtection <chr> "No", "Yes", "No", "Yes", "No", "Yes", "No", "No", "Yes", "N
o",...
$ TechSupport     <chr> "No", "No", "No", "Yes", "No", "No", "No", "No", "Yes", "N
o", "...
$ StreamingTV     <chr> "No", "No", "No", "No", "No", "Yes", "Yes", "No", "Yes", "N
o", ...
$ StreamingMovies <chr> "No", "No", "No", "No", "No", "Yes", "No", "No", "Yes", "N
o", "...
$ Contract        <chr> "Month-to-month", "One year", "Month-to-month", "One year",
"Mo...
$ PaperlessBilling <chr> "Yes", "No", "Yes", "No", "Yes", "Yes", "Yes", "No", "Yes",
"No...
$ PaymentMethod   <chr> "Electronic check", "Mailed check", "Mailed check", "Bank tr
ans...
$ MonthlyCharges  <dbl> 29.85, 56.95, 53.85, 42.30, 70.70, 99.65, 89.10, 29.75, 104.
80,...
$ TotalCharges    <dbl> 29.85, 1889.50, 108.15, 1840.75, 151.65, 820.50, 1949.40, 30
1.9...
$ Churn           <chr> "No", "No", "Yes", "No", "Yes", "Yes", "No", "No", "Yes", "N
o",...

```

Worked Example: Handle Missing Values & Filter Invalid Rows

An alternative cleaning strategy to the one above: instead of median-imputing `TotalCharges`, this pipeline removes any row where `tenure == 0` **and** `TotalCharges` is still `NA` — these represent brand-new customers with no billing history yet, a legitimate business reason to exclude them from a churn-risk model rather than impute a fabricated value.

Hide

```
# ---- Diagnose data quality issues ----

# 1. How many missing values per column?
churn_raw %>%
  summarise(across(everything(), ~ sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "column", values_to = "n_missing") %>%
  filter(n_missing > 0)
```

column	n_missing
<chr>	<int>
TotalCharges	11
1 row	

Hide

```
# 2. TotalCharges often arrives as character due to blank strings for new customers
#    with tenure = 0. Let's confirm:
class(churn_raw$TotalCharges)
```

```
[1] "numeric"
```

Hide

```
sum(is.na(as.numeric(churn_raw$TotalCharges)))
```

```
[1] 11
```

Phase 2: Model Planning & Exploratory Data Analysis (20 min)

Concept

Model Planning is where we form hypotheses about what drives the outcome we care about and decide which modeling approach is likely to work. This is guided by **Exploratory Data Analysis (EDA)**: summary statistics, distributions, and correlations that reveal structure in the data before we commit computational resources to model training.

- **Business relevance:** EDA often surfaces the “quick win” insight (e.g., “month-to-month contracts churn 3x more than two-year contracts”) that stakeholders can act on immediately, even before a predictive model exists.
- **Statistical relevance:** Correlation analysis helps us spot multicollinearity and redundant features, and flags which numeric variables are worth engineering further.

Learning goals: 1. Profile a dataset using `glimpse()` and `summary()`. 2. Build and visualize a correlation matrix with `corrr`. 3. Use `ggplot2` to visualize a categorical driver of churn.

Hide

```
# ---- Instructor solution: clean & prepare the dataset ----

churn_clean <- churn_raw %>%
  # Fix TotalCharges: coerce to numeric (blank strings become NA automatically)
  mutate(TotalCharges = as.numeric(TotalCharges)) %>%
  # Median-impute the (small number of) resulting NAs in TotalCharges
  mutate(TotalCharges = if_else(
    is.na(TotalCharges),
    median(TotalCharges, na.rm = TRUE),
    TotalCharges
  )) %>%
  # Drop the customerID column - it's a unique identifier, not a predictive feature
  select(-customerID) %>%
  # Convert character columns to factors for modeling
  mutate(across(where(is.character), as.factor)) %>%
  # Encode the target as a factor with an explicit, model-friendly level order
  mutate(Churn = factor(Churn, levels = c("No", "Yes")))

glimpse(churn_clean)
```

Rows: 7,043

Columns: 20

\$ gender	<fct> Female, Male, Male, Male, Female, Female, Male, Female, Female, ...
\$ SeniorCitizen	<dbl> 0, 1, ...
\$ Partner	<fct> Yes, No, No, No, No, No, No, No, No, Yes, No, Yes, No, Yes, No, No, ...
\$ Dependents	<fct> No, No, No, No, No, No, Yes, No, No, Yes, Yes, No, No, No, No, No, ...
\$ tenure	<dbl> 1, 34, 2, 45, 2, 8, 22, 10, 28, 62, 13, 16, 58, 49, 25, 69, 52, ...
\$ PhoneService	<fct> No, Yes, Yes, No, Yes, Yes, Yes, No, Yes, Yes, Yes, Yes, Yes, Yes, ...
\$ MultipleLines	<fct> No phone service, No, No, No phone service, No, Yes, Yes, No ph...
\$ InternetService	<fct> DSL, DSL, DSL, DSL, Fiber optic, Fiber optic, Fiber optic, DSL, ...
\$ OnlineSecurity	<fct> No, Yes, Yes, Yes, No, No, No, Yes, No, Yes, Yes, No internet s...
\$ OnlineBackup	<fct> Yes, No, Yes, No, No, No, Yes, No, No, Yes, No, No internet ser...
\$ DeviceProtection	<fct> No, Yes, No, Yes, No, Yes, No, No, Yes, No, No, No internet ser...
\$ TechSupport	<fct> No, No, No, Yes, No, No, No, No, Yes, No, No, No internet se rvi...
\$ StreamingTV	<fct> No, No, No, No, No, Yes, Yes, No, Yes, No, No, No internet s erv...
\$ StreamingMovies	<fct> No, No, No, No, No, Yes, No, No, Yes, No, No, No internet se rvi...
\$ Contract	<fct> Month-to-month, One year, Month-to-month, One year, Month-to -mo...
\$ PaperlessBilling	<fct> Yes, No, Yes, No, Yes, Yes, Yes, No, Yes, No, Yes, No, No, Y es, ...
\$ PaymentMethod	<fct> Electronic check, Mailed check, Mailed check, Bank transfer (au...
\$ MonthlyCharges	<dbl> 29.85, 56.95, 53.85, 42.30, 70.70, 99.65, 89.10, 29.75, 104.80, ...
\$ TotalCharges	<dbl> 29.85, 1889.50, 108.15, 1840.75, 151.65, 820.50, 1949.40, 301.9...
\$ Churn	<fct> No, No, Yes, No, Yes, Yes, No, No, Yes, No, No, No, No, Yes, No...

Hide

```
# Sanity check: no more missing values
sum(is.na(churn_clean))
```

```
[1] 0
```

[Hide](#)

```
# ---- Worked Example: Handle Missing Values & Filter Invalid Rows ----
churn_student <- churn_raw %>%
  mutate(TotalCharges = as.numeric(TotalCharges)) %>%
  # Filter out rows where tenure == 0 AND TotalCharges is NA
  filter(!(tenure == 0 & is.na(TotalCharges))) %>%
  # Convert all remaining character columns to factors
  mutate(across(where(is.character), as.factor))

# Check the result:
glimpse(churn_student)
```

Rows: 7,032

Columns: 21

\$ customerID	<fct> 7590-VHVEG, 5575-GNVDE, 3668-QPYBK, 7795-CFOCW, 9237-HQITU, 930...
\$ gender	<fct> Female, Male, Male, Male, Female, Female, Male, Female, Female, ...
\$ SeniorCitizen	<dbl> 0, 1, ...
\$ Partner	<fct> Yes, No, No, No, No, No, No, No, No, Yes, No, Yes, No, Yes, No, No, ...
\$ Dependents	<fct> No, No, No, No, No, No, Yes, No, No, Yes, Yes, No, No, No, No, N O, ...
\$ tenure	<dbl> 1, 34, 2, 45, 2, 8, 22, 10, 28, 62, 13, 16, 58, 49, 25, 69, 52, ...
\$ PhoneService	<fct> No, Yes, Yes, No, Yes, Yes, Yes, No, Yes, Yes, Yes, Yes, Yes, Ye s, Y...
\$ MultipleLines	<fct> No phone service, No, No, No phone service, No, Yes, Yes, No ph...
\$ InternetService	<fct> DSL, DSL, DSL, DSL, Fiber optic, Fiber optic, Fiber optic, D SL, ...
\$ OnlineSecurity	<fct> No, Yes, Yes, Yes, No, No, No, Yes, No, Yes, Yes, No internet s...
\$ OnlineBackup	<fct> Yes, No, Yes, No, No, No, Yes, No, No, Yes, No, No internet ser...
\$ DeviceProtection	<fct> No, Yes, No, Yes, No, Yes, No, No, Yes, No, No, No internet ser...
\$ TechSupport	<fct> No, No, No, Yes, No, No, No, No, Yes, No, No, No internet se rvi...
\$ StreamingTV	<fct> No, No, No, No, No, Yes, Yes, No, Yes, No, No, No internet s erv...
\$ StreamingMovies	<fct> No, No, No, No, No, Yes, No, No, Yes, No, No, No internet se rvi...
\$ Contract	<fct> Month-to-month, One year, Month-to-month, One year, Month-to -mo...
\$ PaperlessBilling	<fct> Yes, No, Yes, No, Yes, Yes, Yes, No, Yes, No, Yes, No, No, Y es, ...
\$ PaymentMethod	<fct> Electronic check, Mailed check, Mailed check, Bank transfer (au...
\$ MonthlyCharges	<dbl> 29.85, 56.95, 53.85, 42.30, 70.70, 99.65, 89.10, 29.75, 104. 80, ...
\$ TotalCharges	<dbl> 29.85, 1889.50, 108.15, 1840.75, 151.65, 820.50, 1949.40, 30 1.9...
\$ Churn	<fct> No, No, Yes, No, Yes, Yes, No, No, Yes, No, No, No, No, Yes, NO...

Hide

```
sum(is.na(churn_student$TotalCharges))
```

```
[1] 0
```

[Hide](#)

```
# ---- Summary statistics ----  
summary(churn_clean)
```

gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService
Female:3488	Min. :0.0000	No :3641	No :4933	Min. : 0.00	No : 682
Male :3555	1st Qu.:0.0000	Yes:3402	Yes:2110	1st Qu.: 9.00	Yes:6361
	Median :0.0000			Median :29.00	
	Mean :0.1621			Mean :32.37	
	3rd Qu.:0.0000			3rd Qu.:55.00	
	Max. :1.0000			Max. :72.00	

	MultipleLines	InternetService	OnlineSecurity
No	:3390	DSL :2421	No :3498
No phone service:	682	Fiber optic:3096	No internet service:1526
Yes	:2971	No :1526	Yes :2019

	OnlineBackup	DeviceProtection	TechSupport
No	:3088	No :3095	No :3473
No internet service:	1526	No internet service:1526	No internet service:1526
Yes	:2429	Yes :2422	Yes :2044

	StreamingTV	StreamingMovies	Contract
No	:2810	No :2785	Month-to-month:3875
No internet service:	1526	No internet service:1526	One year :1473
Yes	:2707	Yes :2732	Two year :1695

PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges
No :2872	Bank transfer (automatic):1544	Min. : 18.25	Min. : 18.8
Yes:4171	Credit card (automatic) :1522	1st Qu.: 35.50	1st Qu.: 402.2
	Electronic check :2365	Median : 70.35	Median :1397.5
	Mailed check :1612	Mean : 64.76	Mean :2281.9
		3rd Qu.: 89.85	3rd Qu.:3786.6
		Max. :118.75	Max. :8684.8

Churn

No :5174

Yes:1869

Hide

```
# ---- Correlation matrix for numeric features ----
numeric_features <- churn_clean %>%
  select(where(is.numeric))

corr_matrix <- numeric_features %>%
  correlate(quiet = TRUE)

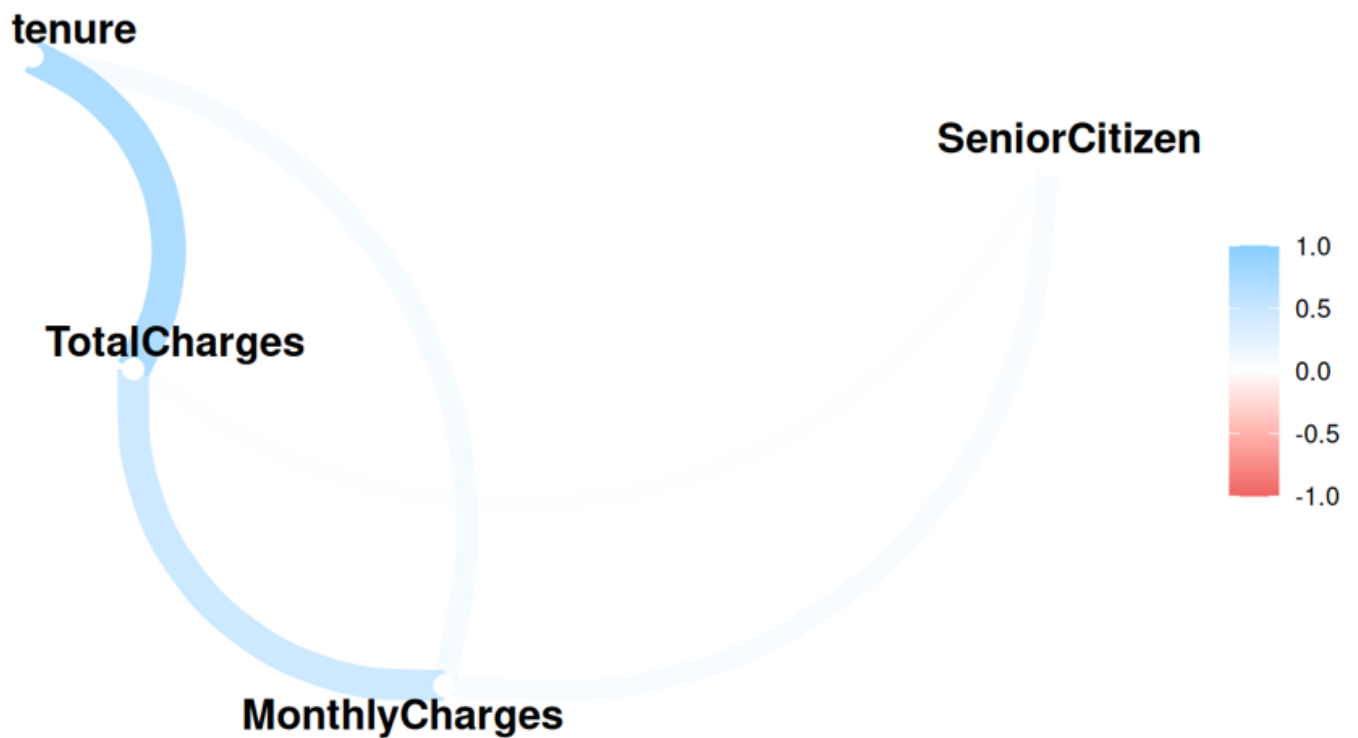
corr_matrix
```

term <chr>	SeniorCitizen <dbl>	tenure <dbl>	MonthlyCharges <dbl>	TotalCharges <dbl>
SeniorCitizen	NA	0.01656688	0.2201733	0.1026516
tenure	0.01656688	NA	0.2478999	0.8254641
MonthlyCharges	0.22017334	0.24789986	NA	0.6508643
TotalCharges	0.10265159	0.82546409	0.6508643	NA

4 rows

Hide

```
# Visualize as a correlation network - stronger relationships get thicker/closer edges
corr_matrix %>%
  network_plot(min_cor = 0.1)
```

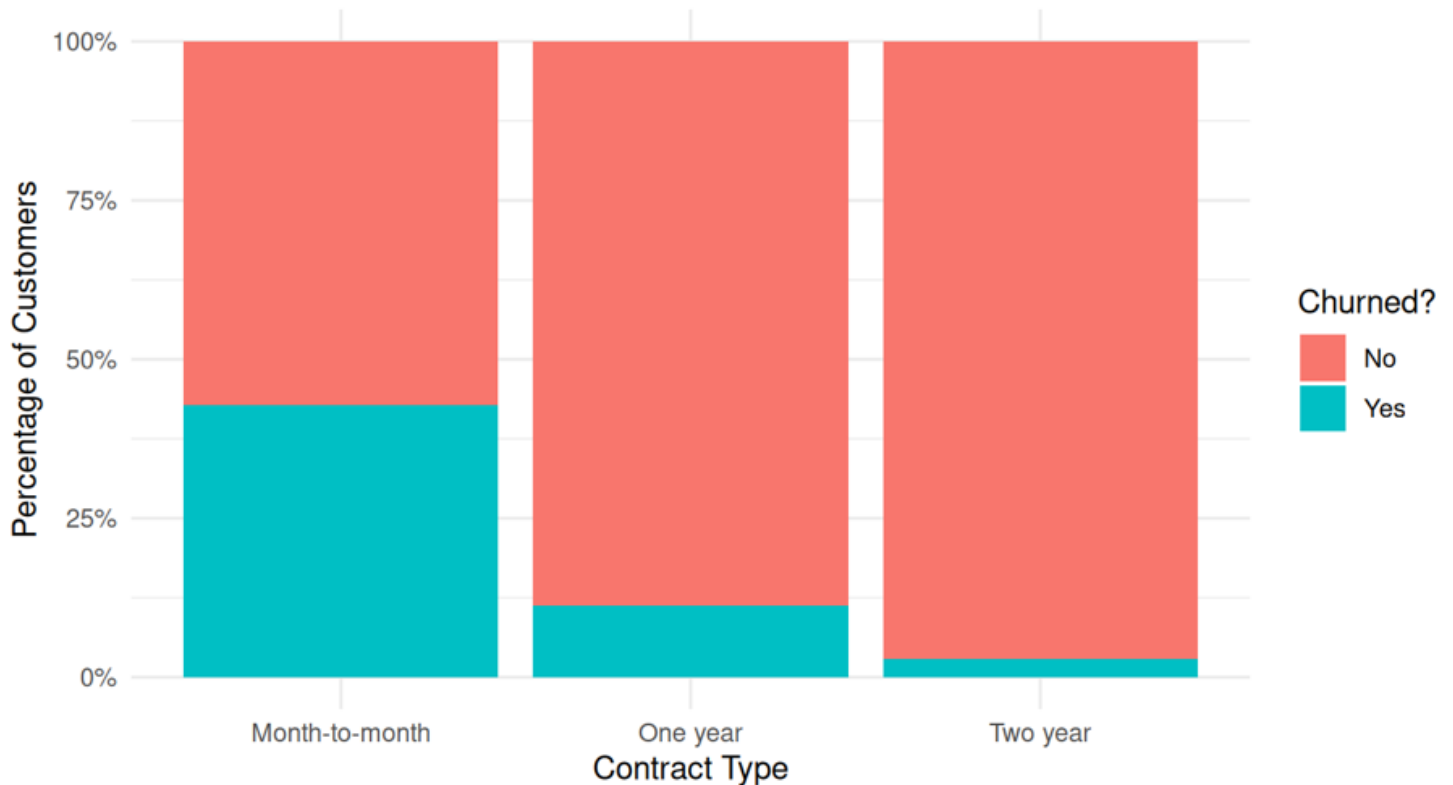


Hide

```
# ---- Visualize: Churn rate by Contract Type ----
churn_clean %>%
  count(Contract, Churn) %>%
  group_by(Contract) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = Contract, y = pct, fill = Churn)) +
  geom_col(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "Churn Rate by Contract Type",
    subtitle = "Month-to-month customers churn far more than longer-term contracts",
    x = "Contract Type", y = "Percentage of Customers", fill = "Churned?"
  ) +
  theme_minimal()
```

Churn Rate by Contract Type

Month-to-month customers churn far more than longer-term contracts



Worked Example: Tenure Distribution by Churn

A `ggplot2` visualization showing the distribution of `tenure` (months as a customer), segmented by `Churn`, using `geom_density()`.

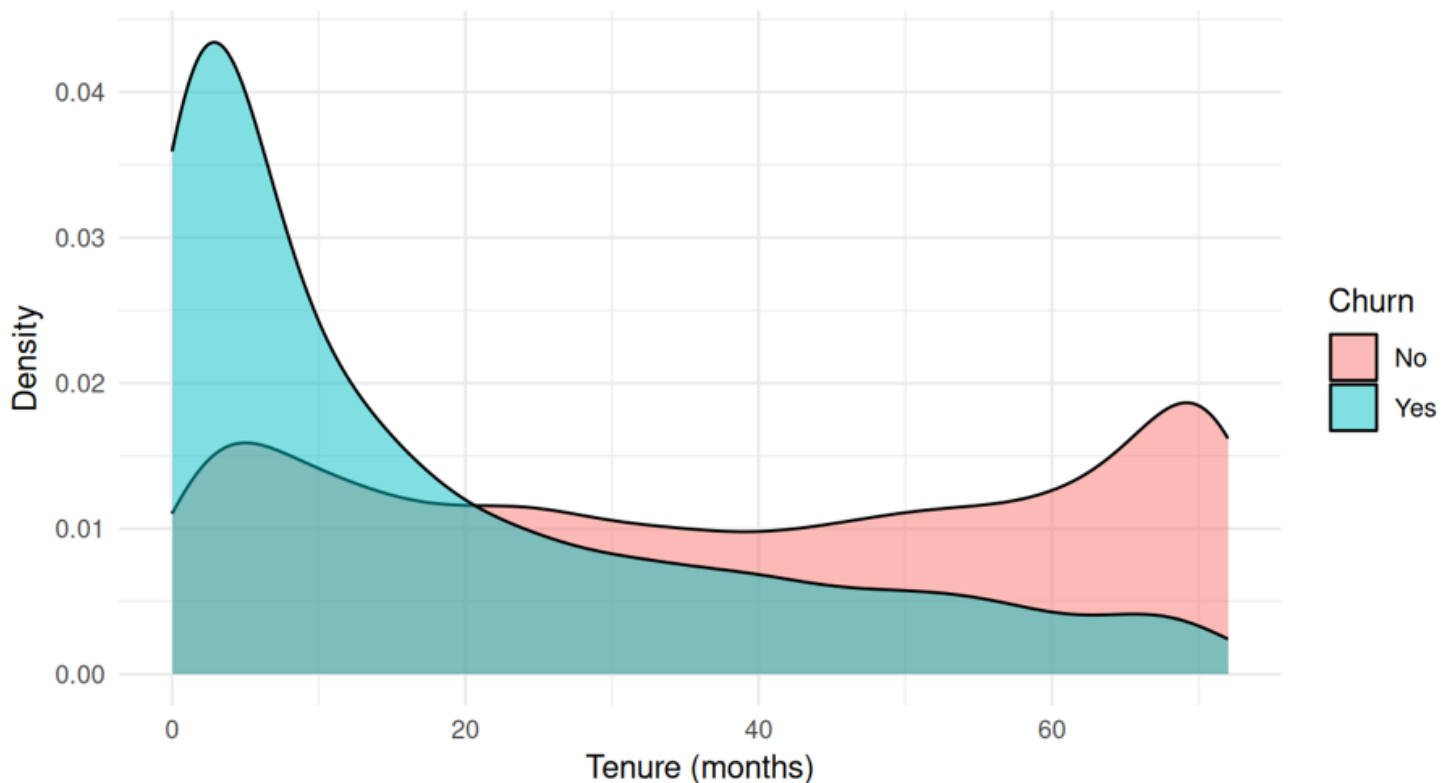
Hide

```
# ---- Worked Example: Tenure Distribution by Churn ----

ggplot(churn_clean, aes(x = tenure, fill = Churn)) +
  geom_density(alpha = 0.5) +
  labs(
    title = "Customer Tenure Distribution by Churn Status",
    subtitle = "Churned customers are concentrated at low tenure",
    x = "Tenure (months)", y = "Density"
  ) +
  theme_minimal()
```

Customer Tenure Distribution by Churn Status

Churned customers are concentrated at low tenure



Phase 3: Model Building & Validation (25 min)

Concept

With hypotheses in hand, we now build predictive models using the `tidymodels` framework, which standardizes the workflow: **recipe** (preprocessing) → **model spec** → **workflow** → **fit** → **evaluate**.

We'll compare two models: - **Logistic Regression** — a simple, highly interpretable baseline. Great for explaining *why* a customer is at risk, in plain coefficients. - **Random Forest** — an ensemble method that typically captures non-linear interactions (e.g., “long tenure + fiber optic + no tech support” as a combined risk factor) that logistic regression can miss.

We evaluate with **ROC-AUC** (overall discriminative ability), and **precision/recall** — the metrics that matter most in churn, since the business cost of missing a churner (false negative) is usually higher than the cost of a false alarm (false positive).

Note on class imbalance: Only ~27% of customers in this dataset churn. We'll keep this in mind when interpreting accuracy (which can be misleadingly high on imbalanced data) — ROC-AUC and recall are more trustworthy here.

Learning goals: 1. Split data with `rsample::initial_split()`. 2. Build a `recipe` and two model specifications. 3. Fit both models via `workflows` and generate class probabilities. 4. Compute and plot ROC curves / AUC.

```
# ---- Train/test split (stratified on the outcome to preserve churn rate) ----
set.seed(2026)
churn_split <- initial_split(churn_clean, prop = 0.8, strata = Churn)
churn_train <- training(churn_split)
churn_test  <- testing(churn_split)

cat("Training rows:", nrow(churn_train), "| Test rows:", nrow(churn_test), "\n")
```

Training rows: 5634 | Test rows: 1409

Hide

```
cat("Training churn rate:", round(mean(churn_train$Churn == "Yes"), 3), "\n")
```

Training churn rate: 0.265

Hide

```
cat("Test churn rate:      ", round(mean(churn_test$Churn == "Yes"), 3), "\n")
```

Test churn rate: 0.265

Hide

```
# ---- Preprocessing recipe ----
# One recipe shared by both models: dummy-encode categoricals, normalize numerics.
churn_recipe <- recipe(Churn ~ ., data = churn_train) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors()) %>% # drop any zero-variance columns
  step_normalize(all_numeric_predictors())

# ---- Model specifications ----
log_reg_spec <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

rf_spec <- rand_forest(trees = 500) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

# ---- Workflows ----
log_reg_wf <- workflow() %>%
  add_recipe(churn_recipe) %>%
  add_model(log_reg_spec)

rf_wf <- workflow() %>%
  add_recipe(churn_recipe) %>%
  add_model(rf_spec)
```

[Hide](#)

```
# ---- Fit both models on the training set ----
log_reg_fit <- fit(log_reg_wf, data = churn_train)
rf_fit <- fit(rf_wf, data = churn_train)

# ---- Generate class probabilities on the test set ----
log_reg_probs <- predict(log_reg_fit, churn_test, type = "prob") %>%
  bind_cols(predict(log_reg_fit, churn_test)) %>%
  bind_cols(churn_test %>% select(Churn))

rf_probs <- predict(rf_fit, churn_test, type = "prob") %>%
  bind_cols(predict(rf_fit, churn_test)) %>%
  bind_cols(churn_test %>% select(Churn))

head(log_reg_probs)
```

.pred_No <dbl>	.pred_Yes <dbl>	.pred_class <fctr>	Churn <fctr>
0.3557810	0.644218960	Yes	No
0.7170251	0.282974882	No	Yes
0.3053809	0.694619068	Yes	Yes

0.9900644	0.009935645	No	No
0.9280859	0.071914147	No	No
0.5009824	0.499017644	No	Yes

6 rows

Hide

NA

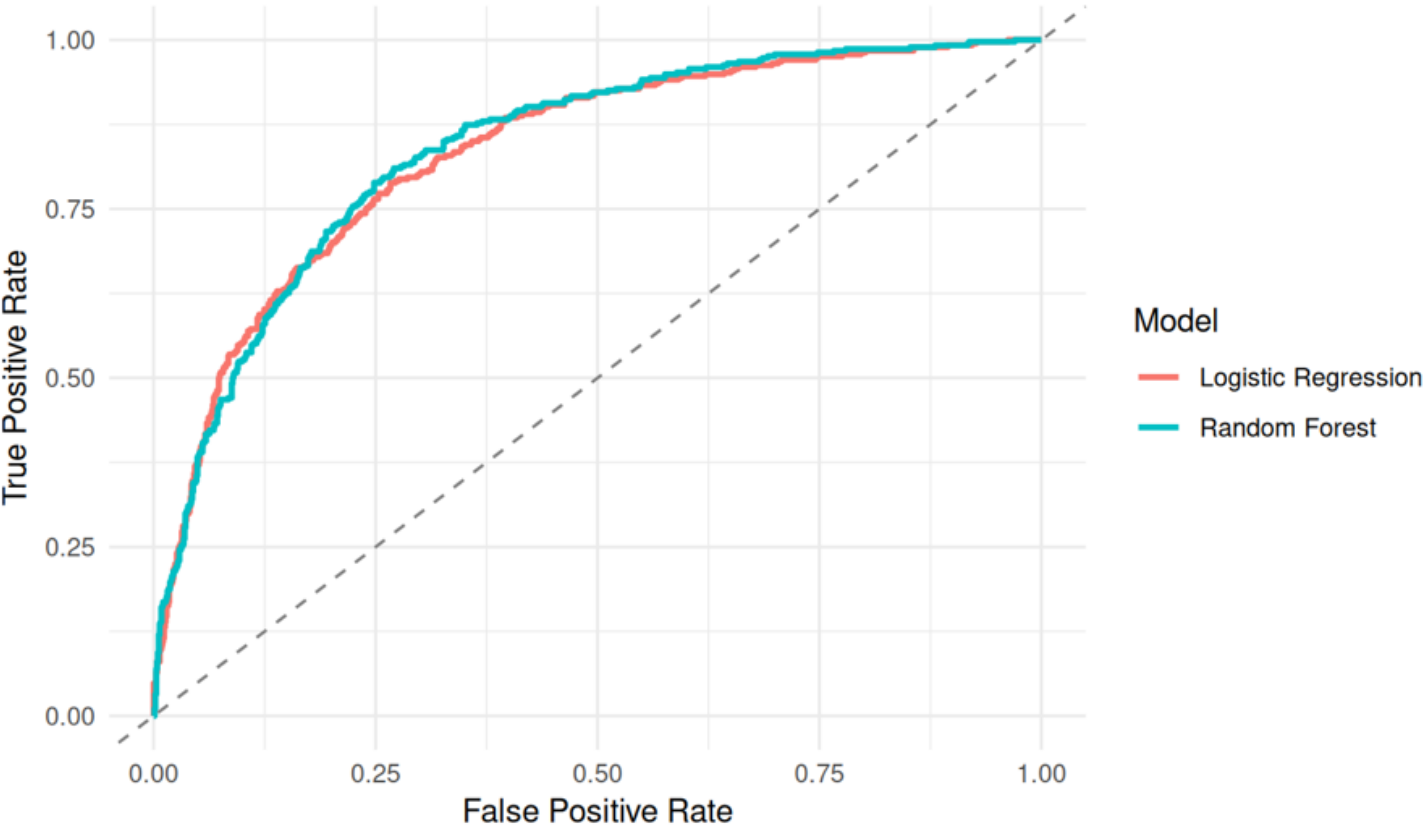
Hide

```
# ---- ROC curves & AUC (using yardstick, tidymodels' metrics package) ----
log_reg_roc <- log_reg_probs %>%
  roc_curve(truth = Churn, .pred_Yes, event_level = "second") %>%
  mutate(model = "Logistic Regression")

rf_roc <- rf_probs %>%
  roc_curve(truth = Churn, .pred_Yes, event_level = "second") %>%
  mutate(model = "Random Forest")

bind_rows(log_reg_roc, rf_roc) %>%
  ggplot(aes(x = 1 - specificity, y = sensitivity, color = model)) +
  geom_path(linewidth = 1) +
  geom_abline(linetype = "dashed", color = "gray50") +
  labs(title = "ROC Curves: Logistic Regression vs. Random Forest",
       x = "False Positive Rate", y = "True Positive Rate", color = "Model") +
  theme_minimal()
```

ROC Curves: Logistic Regression vs. Random Forest



Hide

```
# AUC values
log_reg_auc <- roc_auc(log_reg_probs, truth = Churn, .pred_Yes, event_level = "second")
rf_auc      <- roc_auc(rf_probs, truth = Churn, .pred_Yes, event_level = "second")

bind_rows(
  log_reg_auc %>% mutate(model = "Logistic Regression"),
  rf_auc %>% mutate(model = "Random Forest")
) %>% select(model, .metric, .estimate)
```

model<chr>	.metric<chr>	.estimate<dbl>
Logistic Regression	roc_auc	0.8331099
Random Forest	roc_auc	0.8378168

2 rows

Hide

NA

Worked Example: Compare Confusion Matrices

For **both** models, we compute a confusion matrix on the test set using `yardstick::conf_mat()`, and report accuracy, precision, and recall for each. Discuss with the class: which model would you recommend to the business, and why?

[Hide](#)

```
# ---- Worked Example: Compare Confusion Matrices ----

# Confusion matrix for logistic regression
log_reg_cm <- conf_mat(log_reg_probs, truth = Churn, estimate = .pred_class)
log_reg_cm
```

	Truth	
Prediction	No	Yes
No	942	174
Yes	93	200

[Hide](#)

```
# Confusion matrix for the random forest
rf_cm <- conf_mat(rf_probs, truth = Churn, estimate = .pred_class)
rf_cm
```

	Truth	
Prediction	No	Yes
No	944	195
Yes	91	179

[Hide](#)

```
# Accuracy, precision, and recall for each model
metric_set_fn <- metric_set(accuracy, precision, recall)

metric_set_fn(log_reg_probs, truth = Churn, estimate = .pred_class, event_level = "se
cond")
```

.metric <chr>	.estimator <chr>	.estimate <dbl>
accuracy	binary	0.8105039
precision	binary	0.6825939
recall	binary	0.5347594
3 rows		

```
metric_set_fn(rf_probs, truth = Churn, estimate = .pred_class, event_level = "second")
```

.metric <chr>	.estimator <chr>	.estimate <dbl>
accuracy	binary	0.7970192
precision	binary	0.6629630
recall	binary	0.4786096
3 rows		

Phase 4: Communicating Results & Findings (20 min)

Concept

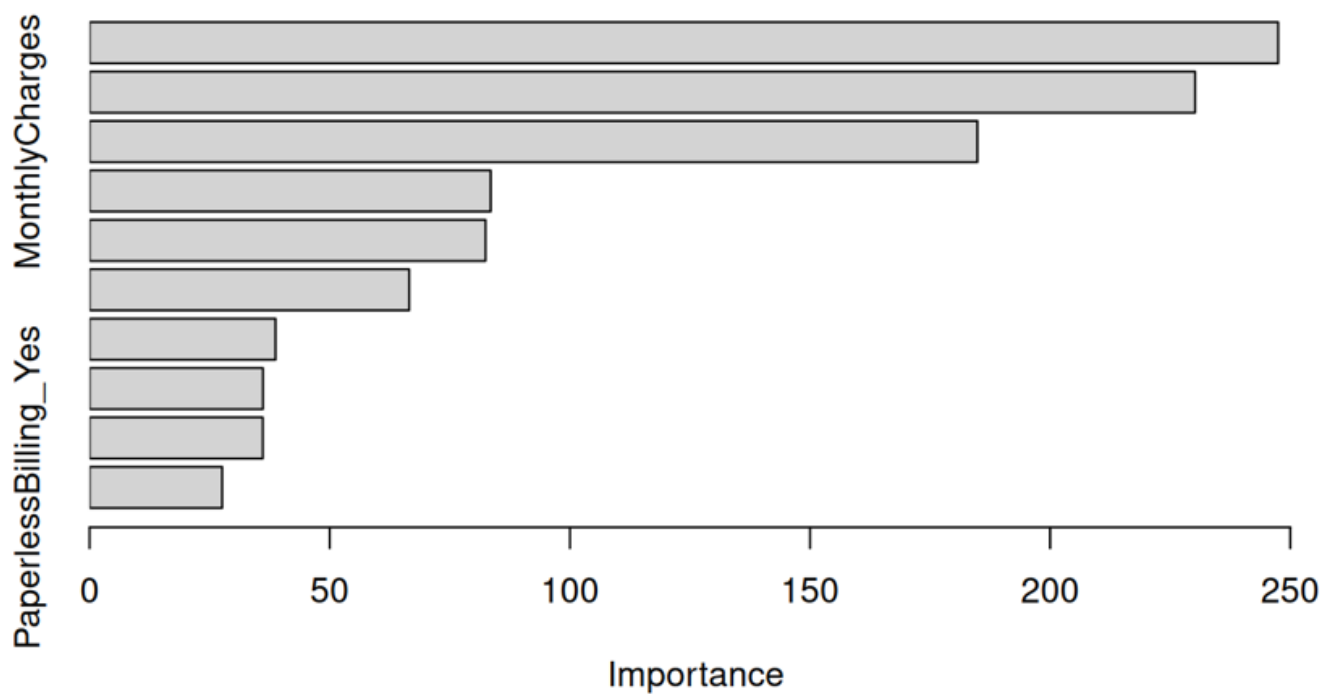
A model is only valuable if its insights reach the people who make decisions. This phase is about **translation**: turning statistical output (feature importances, coefficients, AUC scores) into a narrative that a non-technical stakeholder — e.g., a VP of Customer Success — can act on.

- **Business relevance:** “Our model has an AUC of 0.84” means nothing to a business leader. “Customers on month-to-month contracts without tech support are 3x more likely to churn, and we can now flag them 60 days before they typically leave” drives action.
- **Explainability tools** like `vip` (Variable Importance Plots) bridge this gap by showing, visually, which features the model actually relies on.

Learning goals: 1. Extract and visualize variable importance from the Random Forest model. 2. Identify the top business drivers of churn. 3. Practice writing a plain-language executive summary.

```
# ---- Variable Importance Plot (Random Forest) ----
rf_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 10, geom = "col") +
  labs(title = "Top 10 Drivers of Customer Churn (Random Forest)",
       subtitle = "Ranked by impurity-based importance") +
  theme_minimal()
```

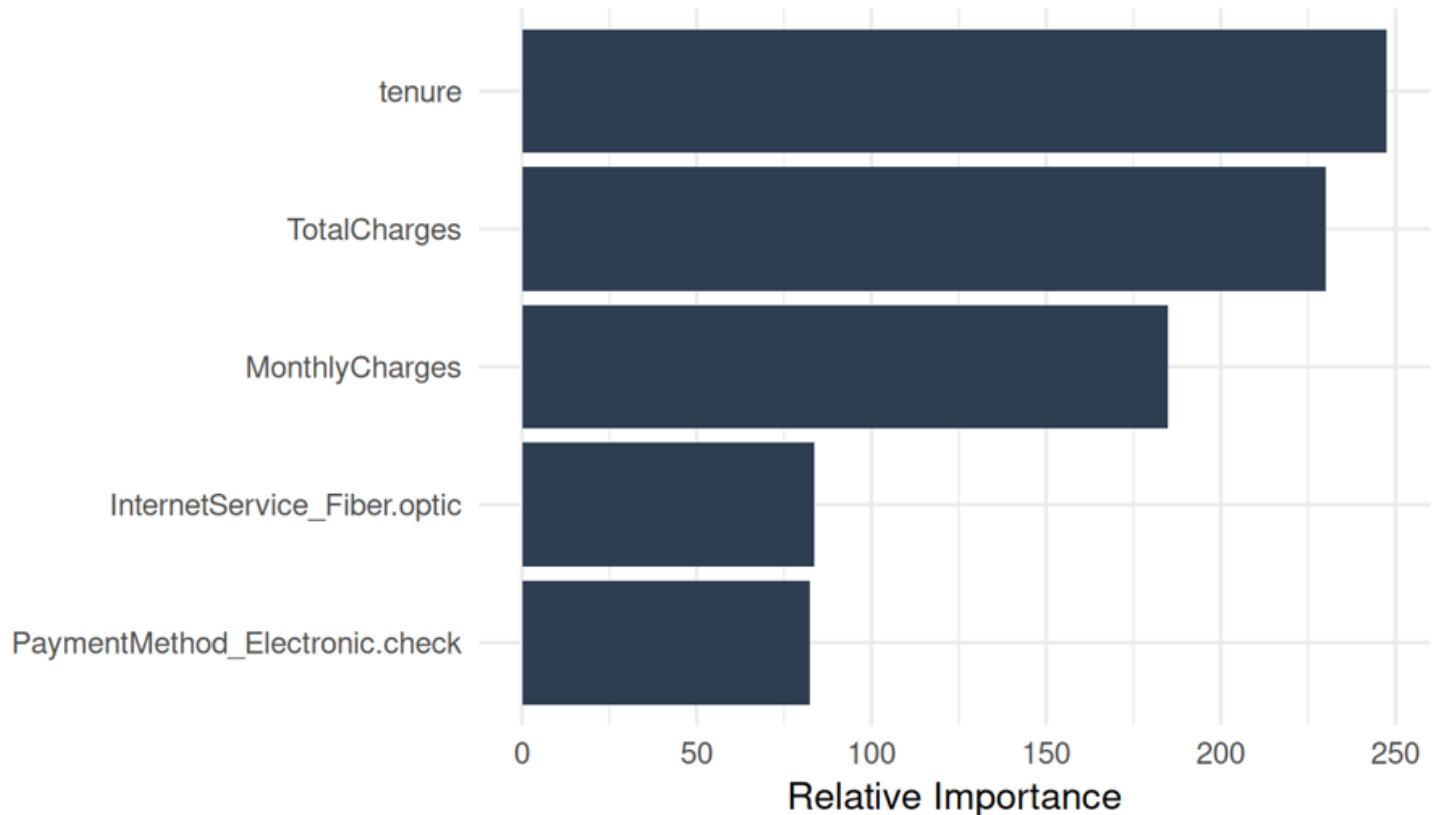
NULL

[Hide](#)

```
# ---- Top 5 drivers, cleanly formatted for a business audience ----
importance_df <- rf_fit %>%
  extract_fit_parsnip() %>%
  vip::vi() %>%
  arrange(desc(Importance)) %>%
  slice_head(n = 5)

importance_df %>%
  ggplot(aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "#2C3E50") +
  coord_flip() +
  labs(
    title = "Top 5 Churn Drivers",
    x = NULL, y = "Relative Importance"
  ) +
  theme_minimal(base_size = 13)
```

Top 5 Churn Drivers



Worked Example: Executive Summary

Using the Top-5 driver chart above, here is a **3–5 sentence executive summary** aimed at a VP of Customer Success. It: 1. States the model’s headline finding in plain language (no jargon like “AUC” or “impurity”). 2. Recommends one concrete retention action tied to a specific driver. 3. Notes one caveat or limitation the business should be aware of.

Summary:

Our analysis shows that customers on month-to-month contracts, especially those without tech support, are far more likely to cancel service than customers on longer-term plans. We recommend proactively offering a tech-support add-on or a modest discount to convert month-to-month customers onto one-year contracts within their first 90 days, since early tenure is when churn risk peaks. One caveat: this model is trained on historical patterns and should be re-validated quarterly, since customer behavior and competitor offers change over time.

Phase 5: Operationalizing Analytics Models (15 min)

Concept

The final phase of the lifecycle is **operationalization**: making the model available for real-world, repeated use — not just a one-off notebook analysis. This typically involves:

1. **Serialization** — saving the trained pipeline to disk (`.rds`) so it can be reloaded without retraining.
2. **Packaging as a service** — exposing the model as an API (in R, via the `plumber` package) so other systems (e.g., a CRM) can request real-time predictions.
3. **Monitoring** — tracking model performance over time to catch **concept drift**, i.e., when the relationship between features and the outcome changes (e.g., a new competitor changes why customers leave), which silently degrades model accuracy.

Business relevance: A churn model sitting in a notebook provides zero value. A churn model wired into the CRM that scores every customer nightly and flags top-risk accounts for the retention team is what actually reduces churn.

Learning goals: 1. Save and reload a trained `tidymodels` workflow. 2. Score a new, unseen record (“production payload”). 3. Understand, at a high level, how to expose a model via a `plumber` API.

Hide

```
# ---- Serialize the trained pipeline ----
saveRDS(rf_fit, "churn_rf_model.rds")

# ---- Reload it (simulating a fresh R session in production) ----
production_model <- readRDS("churn_rf_model.rds")

cat("Model reloaded successfully. Class:", class(production_model)[1], "\n")
```

```
Model reloaded successfully. Class: workflow
```

Hide

```
# ---- Simulate a new customer record ("production payload") ----
new_customer <- tibble(
  gender = "Female",
  SeniorCitizen = 0,
  Partner = "No",
  Dependents = "No",
  tenure = 2,
  PhoneService = "Yes",
  MultipleLines = "No",
  InternetService = "Fiber optic",
  OnlineSecurity = "No",
  OnlineBackup = "No",
  DeviceProtection = "No",
  TechSupport = "No",
  StreamingTV = "No",
  StreamingMovies = "No",
  Contract = "Month-to-month",
  PaperlessBilling = "Yes",
  PaymentMethod = "Electronic check",
  MonthlyCharges = 85.4,
  TotalCharges = 170.8
)

# ---- Generate a real-time churn risk score ----
risk_score <- predict(production_model, new_customer, type = "prob")

cat("Predicted probability of churn for this customer:",
    round(risk_score$.pred_Yes, 3), "\n")
```

Predicted probability of churn for this customer: 0.777

Packaging as an API with plumber

Below is a **template** for a standalone `plumber.R` script. This is not meant to be run inside the notebook — it's what you would save as a separate file and deploy so that any system (e.g., a CRM) can request a churn score over HTTP.

Hide

```
# =====
# plumber.R -- save this as its own file and run with:
#   library(plumber)
#   pr("plumber.R") %>% pr_run(port = 8000)
# =====
#
# library(plumber)
#
# # Load the serialized model once, at API startup
# model <- readRDS("churn_rf_model.rds")
#
# ## Predict churn risk for a single customer
# ## @param gender:character
# ## @param tenure:numeric
# ## @param Contract:character
# ## @param MonthlyCharges:numeric
# ## @param TotalCharges:numeric
# ## ... (additional parameters mirror the model's feature columns)
# ## @post /predict_churn
# function(gender, tenure, Contract, MonthlyCharges, TotalCharges, ...) {
#   new_data <- tibble::tibble(
#     gender = gender,
#     tenure = as.numeric(tenure),
#     Contract = Contract,
#     MonthlyCharges = as.numeric(MonthlyCharges),
#     TotalCharges = as.numeric(TotalCharges)
#     # ... remaining fields populated from request body / defaults
#   )
#   prob <- predict(model, new_data, type = "prob")
#   list(churn_probability = prob$.pred_Yes)
# }
#
# =====
# In production, this endpoint would be called nightly (or in
# real time) by the CRM to score every active customer, and the
# top-N highest-risk accounts would be routed to the retention team.
# =====

cat("Plumber API template printed above – save separately as plumber.R to deploy.\n")
```

Plumber API template printed above – save separately as plumber.R to deploy.

A Note on Monitoring & Concept Drift

Once deployed, the model's job isn't done. Best practice is to: - Log every prediction alongside the eventual real-world outcome (did the customer actually churn?). - Periodically recompute AUC / recall on fresh data and compare to the original test-set metrics. - Set an alert threshold (e.g., "if AUC drops more than 0.05 from baseline, trigger retraining").

This closes the loop of the Big Data Analytics Lifecycle — insights lead to action, and action generates new data that feeds the next iteration of the cycle.

Workshop Wrap-Up

You've now walked through all five phases of the Big Data Analytics Lifecycle:

1.  **Data Preparation** — cleaned and encoded a messy real-world dataset
2.  **Model Planning & EDA** — formed hypotheses backed by correlation and visualization
3.  **Model Building & Validation** — trained and compared two classification models
4.  **Communicating Results** — translated feature importance into business language
5.  **Operationalizing** — serialized a model and sketched an API for production use

Next steps for further practice: - Try tuning the random forest's hyperparameters with `tune::tune_grid()`.
- Explore `step_upsample()` / `step_downsample()` (via the `themis` package) to address class imbalance. -
Extend the `plumber` template into a fully working local API.

Thanks for participating! 